

Beyond Traditional KGQA: What LLMs Are For, and Why the Field Is Genuinely Hard

Bernard Katamanso

August 1, 2026

1 What Do We Actually Want LLMs For?

The common failure mode is to treat the LLM as a *knowledge store*. It is not: parametric memory is frozen at training time, unverifiable, and hallucinates precisely where facts are domain-specific. The useful reframing is a *division of labor* between a linguistic system and a structured system.

Role	Performed by	Why the LLM
Comprehend	LLM	Turn noisy natural language into intent, entity mentions, and constraints
Compose	LLM + symbolic layer	Plan/execute multi-hop reasoning: the LLM <i>proposes</i> , structure <i>disposes</i>
Communicate	LLM	Produce the answer plus explanation in natural language

The knowledge graph is the ground truth; the LLM is the *linguistic front-end* and *reasoning orchestrator*—never the memory. This is the neuro-symbolic split, and DCA-Trie instantiates it: the LLM generates, the TypeOracle and trie decide what is legal.

The LLM earns its place exactly when the question is **not answerable by lookup alone**: when it needs composition, path-finding, or a natural-language interface to structured data. Users of a substation or hospital system will not write SPARQL; they will say “which feeder is feeding the faulted bus?” That utterance requires comprehension, decomposition, and path-finding over a structured model.

2 RAG vs. Agentic Retrieval vs. Constrained Decoding

These are three points on the **fluency–fidelity tradeoff**: how tightly the model’s output is bound to external structure.

2.1 Retrieval-Augmented Generation (RAG)

- *Context* is grounded: retrieve relevant passages/subgraph, then generate conditioned on them.
- The *generation* remains free: the model can still ignore the retrieved context and answer from parametric memory.
- One-shot, no feedback loop.
- Retrieval quality is the bottleneck: bad retrieval $\Rightarrow$ bad answer, regardless of generation quality.

2.2 Agentic Retrieval

- Iterative, tool-using, reflective: the model decides what to fetch, observes the results, and re-plans.
- Buys **backtracking and recovery**—the structural capability that a single-pass decoder lacks.
- Examples: KBQA-o1 (MCTS), GraphRAG-style agent loops.
- Cost: reasoning loops can explode; tool selection is still model-decided, so the agent can chase the wrong subgraph.

2.3 Constrained Decoding

- The tightest coupling: the graph constrains the *generation process itself* at the token level.
- RAG *suggests*; constrained decoding *forbids*. It is impossible for the model to emit a token that leads off the graph.
- The strongest fidelity guarantee—at the cost of **dead ends**: over-constraint can leave no valid completion.

	RAG	Agentic	Constrained
Grounding point	Context	Process	Tokens
Feedback loop	No	Yes	No
Backtracking	No	Yes	No
Can hallucinate off-graph	Yes	Yes	No
Risk	Stale/irrelevant context	Loop explosion	Dead ends

The interesting design space is *combining* them: agentic planning on the outside, constrained decoding on the inside.

3 You Can Define Your Own Schema—and That Is the Payoff

This is the key reason the TypeOracle generalizes beyond Freebase. Freebase is a *general, shallow* ontology, which is exactly why our gates are permissive (13% path reduction). A **domain schema** is **narrow, deep, and precise**, so semantic gates become much stronger.

3.1 Substation / Electrical Grid (IEC 61850, IEC 61970)

- **Entities:** transformers, breakers, buses, relays, protection zones, feeders.
- **Relations:** feeds, protects, measures, connected_to.
- Example gate: $\text{range}(\text{protects}) = \text{protection devices} \rightarrow \text{protection zones}$. The path $\text{transformer} \rightarrow \text{protects} \rightarrow \text{bus}$ is rejected because a transformer is not a protection device. In a safety-critical domain, that rejection is the whole point.

3.2 Health (SNOMED CT, RxNorm, ICD)

- **Entities:** patients, conditions, drugs, genes, proteins, trials.
- **Relations:** treats, contraindicates, associated_with.
- Example gate: a **contraindicates** gate prevents pairing a drug with an incompatible condition.

3.3 The Living Graph

A substation’s SCADA telemetry is an event stream that *mutates* the graph: breakers open, loads change, zones reconfigure. The constrained-decoding approach survives this because the trie is rebuilt from the **current snapshot** for each query. The questions become temporal—“what is the state of the network *now*?”—which is temporal KGQA, a regime where a frozen parametric model fundamentally cannot compete.

That the schema is *yours* is also the point: Freebase constrains weakly because it is generic; your domain ontology constrains strongly because you control it.

4 Why the Field Is Genuinely Hard

Seven fundamental barriers—not engineering issues.

4.1 1. The representation gap

Natural language is linear, ambiguous, and under-specified; a graph is exact and combinatorial. Mapping “the big fuse box thing” to **transformer #7** is entity linking, and it is genuinely unsolved for messy input.

4.2 2. Combinatorial reasoning

Multi-hop reasoning is exponential: $O(d^L)$ paths. Every constraint that makes this tractable trades away recall. This is an information-theoretic tension, not a tuning problem.

4.3 3. Open-world incompleteness

KGs miss edges, miss types, and contain aliases and errors. The oracle’s conservatism (“allow when unknown”) is a double-edged sword: it never wrongly filters, but it barely filters at all.

4.4 4. Ontology quality is the ceiling

Semantic gates are only as strong as the type system. Coarse ontology $\Rightarrow$ weak pruning (our 13% on Freebase). This is a *data* problem, not an algorithm problem.

4.5 5. Hallucination is the model’s inductive bias

LLMs are trained to be fluent, and fluency means “plausible continuation”—not “faithful to the graph.” Constraining generation fights the training objective itself. This is why constrained decoding is genuinely harder than it sounds.

4.6 6. Living knowledge

Static tries and embeddings go stale. Reasoning over changing graphs (temporal KGQA) is largely unsolved.

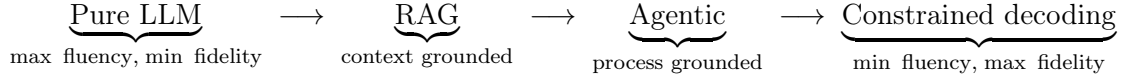
4.7 7. Trust, not accuracy

89% Hits@1 is a publication result; it is *unacceptable* for a hospital or a grid operator. Those domains demand provable, citable, explainable answers—and the ability to say “I do not know” rather than guess. The entire fidelity apparatus exists to serve *verifiability*, and current evaluation metrics do not even measure it.

5 Synthesis

The field is hard because it asks a **probabilistic, ungrounded system** to behave like a **deterministic, grounded query engine**—while preserving the fluency that makes the probabilistic system useful in the first place.

Every method is a point on the fluency–fidelity tradeoff:



DCA-Trie’s thesis is that one can have *a lot* of both by constraining at the token level with a *semantic* gate—not just a topological one—and especially by controlling the schema. The concrete move from 13% Freebase pruning to a 60%+ reduction on a domain ontology is the empirical path forward: the ceiling on pruning is set by the ontology, and the ontology is now ours to build.